



Rune

AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING
COMPUTER AND SYSTEMS ENGINEERING PROGRAM

RUNE
Low Code Automation Platform
Technical Documentation

Academic Year 2025–2026

Prepared by:

Shehab Mahmoud Abdelrahman Hany
Youssef Shahean Seif Tamer
Seif Elwarwary Omar Mamon
Habiba El Sayed Zeynat Haytham
Shahd Ashraf

Supervised by:

Dr. Yomna Safaaeldin
Eng. Amr Ibrahim

December 23, 2025

Abstract

Modern life and organizational environments are increasingly dominated by repetitive daily routines, both personal and professional. From routine administrative work such as sending emails, responding to customer inquiries, scheduling tasks, and managing data, to operational workflows that require constant manual intervention, a significant amount of time is consumed by activities that add little strategic value. This continuous repetition not only wastes time but also limits productivity and prevents individuals and organizations from focusing on higher-level decision-making and innovation.

The emergence of artificial intelligence and automation presents an opportunity to rethink how such tasks are performed. By delegating repetitive processes to intelligent systems, time can be reclaimed and utilized more effectively.

RUNE addresses this challenge by providing a low-code, AI-powered workflow automation platform that enables users to design, execute, and manage automated workflows with minimal technical effort. Through an intuitive interface, users can describe their workflows in natural language, connect multiple services and APIs, and automate repetitive tasks across various platforms. The system leverages artificial intelligence to intelligently orchestrate workflow execution, handle complex decision logic, and adapt to dynamic conditions, allowing humans to focus on creativity, strategy, and growth. RUNE provides visual workflow design capabilities while enhancing the experience with AI-driven assistance for workflow creation, optimization, and troubleshooting.

“The bad news is time flies. The good news is you’re the pilot.”

— *Michael Altshuler*

Contents

Abstract	i
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Project Objectives	1
1.4 Project Scope	2
2 System Architecture	3
2.1 High-Level Architecture	3
2.1.1 Components Overview	4
2.2 Technology Stack	5
2.2.1 Why FastAPI?	5
2.2.2 Why Go?	5
2.2.3 Why Rust?	5
2.2.4 Why RabbitMQ?	5
2.2.5 Why PostgreSQL?	6
2.2.6 Why Redis?	6
2.3 Communication Flow	6
2.4 Workflow DSL Structure	6
2.4.1 Core DSL Components	7
2.4.2 Supported Node Types	8
2.4.3 Parameter Resolution	9
2.4.4 Error Handling	9
2.4.5 Schema Validation	9
3 Core Components	11
3.1 Frontend Application	11
3.1.1 User Interface Components	11
3.1.2 WebSocket Client Integration	15
3.1.3 State Management	15
3.2 Backend Master (Python - FastAPI)	16
3.2.1 Overview	16
3.2.2 Design and Responsibilities	16
3.2.3 Key Capabilities	16
3.2.4 Workflow Lifecycle (high level)	16
3.2.5 Resilience and Operational Considerations	17

3.2.6	Extensibility	17
3.3	Queues (RabbitMQ)	17
3.3.1	Global exchange and conventions	17
3.3.2	runs	18
3.3.3	status	18
3.3.4	completion	18
3.3.5	token	18
3.4	RUNE Worker Engine (Go)	18
3.4.1	Overview	18
3.4.2	Architecture and Design	18
3.4.3	Core Components	19
3.4.4	Key Capabilities	19
3.4.5	Execution Flow	20
3.4.6	Operational Characteristics	20
3.4.7	Message Protocols	21
3.4.8	Implementation Structure	21
3.5	RUNE Token Execution Service (RTES) - WebSocket Server (Rust)	21
3.5.1	Overview	21
3.5.2	Purpose and Responsibilities	21
3.5.3	Architecture and Design	22
3.5.4	Key Capabilities	22
3.5.5	Integration with Backend Systems	22
3.5.6	User Experience Benefits	23
3.6	Redis Cache Layer	23
3.6.1	Overview	23
3.6.2	Role Across System Components	23
3.6.3	Key Functions	24
3.6.4	Reliability and Data Consistency	24
3.6.5	Performance Benefits	24
3.7	AI Integration Components	24
3.7.1	Overview	24
3.7.2	Smith - Workflow Generator	25
3.7.3	Scryb - Documentation Generator	25
3.7.4	Integration with RUNE Platform	26
4	Data Flow & Execution Lifecycle	27
4.1	Overview	27
4.2	High-level Execution Flow	27

4.3	Message and Data Lifecycles	28
4.4	Wait / Resume Semantics	28
4.5	Error Handling and Retry Behavior	29
4.6	Real-time Updates and RTES Integration	29
4.7	Observability and Auditing	29
5	Security	31
5.1	Authentication and Authorization	31
5.2	Secrets and Credentials Handling	31
5.3	Workflow Execution and Runtime Protections	31
5.4	Sharing Strategies	32
5.5	Credential Sharing and Workflow Export	32
6	Implementation Challenges & Solutions	33
6.1	Secrets Management and Exporting	33
6.2	Wait Nodes and Scheduling Triggers (Scalable Timers)	34
6.3	Schema and Configuration Consistency	35
6.4	RTES: Execution Logs and Live Updates	35
6.5	Additional Operational Challenges	36
7	Testing Strategy	37
7.1	Frontend Testing	37
7.2	Unit Testing	37
7.3	Integration Testing	37
7.4	AI Testing	38
7.5	Performance Testing	38
8	Deployment & DevOps	39
8.1	Containerization	39
8.2	Docker Compose and Local Orchestration	39
8.3	CI/CD Pipelines	39
8.4	Building	40
8.4.1	Prerequisites	40
8.4.2	Environment Configuration	40
8.4.3	Building with Makefile (Recommended)	40
8.4.4	Building with Docker Compose	41
8.4.5	Cleanup	41

9	Future Enhancements	42
9.1	Platform Features	42
9.2	Integrations	42
9.3	Enterprise Features	42
9.4	Performance	42
10	Conclusion	43
11	References	43
11.1	Project Resources	43
11.2	Core Technologies	43
11.3	Development Tools	44
11.4	Observability and Monitoring	45
	Acknowledgments	45
	Appendices	46
	Appendix A: API Reference	46
	Appendix B: Message Schemas	46
	Appendix C: Database Schema	46
	Appendix D: Configuration Reference	47
	Appendix E: Deployment Guide	47

List of Figures

1	RUNE System Architecture Overview	4
2	Home Page	12
3	Dashboard page	13
4	Canvas Page	14
5	Real-time Workflow Execution	15

1 Introduction

The rapid growth of digital systems and online services has significantly increased the complexity of daily operational workflows across both personal and organizational environments. Tasks such as data synchronization, email communication, customer support handling, notifications, and scheduled operations are often repetitive in nature yet require continuous manual execution. As a result, valuable time and human effort are consumed by routine activities rather than strategic planning, innovation, and problem-solving. Workflow automation has emerged as a key solution to address these challenges by enabling systems to execute predefined tasks with minimal human intervention.

1.1 Background and Motivation

Existing workflow automation platforms have demonstrated the potential to reduce manual effort and improve operational efficiency. However, many of these solutions require advanced technical expertise, complex configurations, or extensive scripting, making them inaccessible to non-technical users. Additionally, limited flexibility, poor user experience, and insufficient intelligence in handling dynamic workflows remain common limitations. The motivation behind RUNE is to bridge this gap by providing an intuitive, low-code automation platform that combines ease of use with the power of artificial intelligence to manage complex workflows effectively.

1.2 Problem Statement

Despite the availability of automation tools, several technical and practical challenges persist. Current solutions often suffer from steep learning curves, rigid workflow structures, limited support for intelligent decision-making, and difficulties in integrating heterogeneous services. These barriers prevent widespread adoption and restrict automation to technically skilled users. Furthermore, the lack of AI-driven adaptability results in workflows that are unable to respond dynamically to varying inputs, changing conditions, or user intent.

1.3 Project Objectives

The primary objective of the RUNE platform is to design and implement a low-code, AI-powered workflow automation system that simplifies the creation, execution, and management of automated workflows. The project aims to:

- Reduce the time and effort required to automate repetitive tasks.

- Enable both technical and non-technical users to design workflows through an intuitive interface.
- Incorporate artificial intelligence to enhance workflow intelligence and adaptability.
- Support seamless integration with external services and APIs.
- Improve overall productivity and user experience.

1.4 Project Scope

The scope of this project includes the design and development of the core workflow engine, low-code workflow definition mechanisms, AI-assisted automation features, and integration capabilities with selected external services. The platform focuses on automating common repetitive tasks such as data handling, notifications, and service coordination. Advanced enterprise features such as large-scale distributed orchestration, custom AI model training, and industry-specific compliance mechanisms are considered outside the scope of this project.

2 System Architecture

2.1 High-Level Architecture

RUNE follows a Master–Worker, message-driven architecture optimized for low end-to-end latency and predictable throughput. The Backend Master (FastAPI) is the authoritative control plane: it validates requests, resolves credentials, persists execution records, and publishes compact execution payloads to the message bus. Lightweight, stateless Worker Engine instances (Go) consume node execution messages from RabbitMQ and perform node work concurrently. The Real-Time Execution Service (RTES, Rust) receives status events via Redis and broadcasts them over WebSocket to connected clients for minimal-latency UI updates. PostgreSQL holds authoritative state and artifacts; Redis provides ephemeral state, continuation storage, caching, and pub/sub; RabbitMQ provides reliable routing, DLQs, and message durability.

Design principles:

- Minimize per-message size and serialization overhead to reduce network and queue latency.
- Keep workers stateless so instances can scale horizontally and recover via message redelivery.
- Use compact, single-responsibility services (Master, Worker, RTES) to reduce tail latency.
- Preserve durability and observability: persistent execution records in PostgreSQL and structured status messages for tracing and replay.
- Prefer async, non-blocking I/O and efficient concurrency models to maximize throughput with low latency.

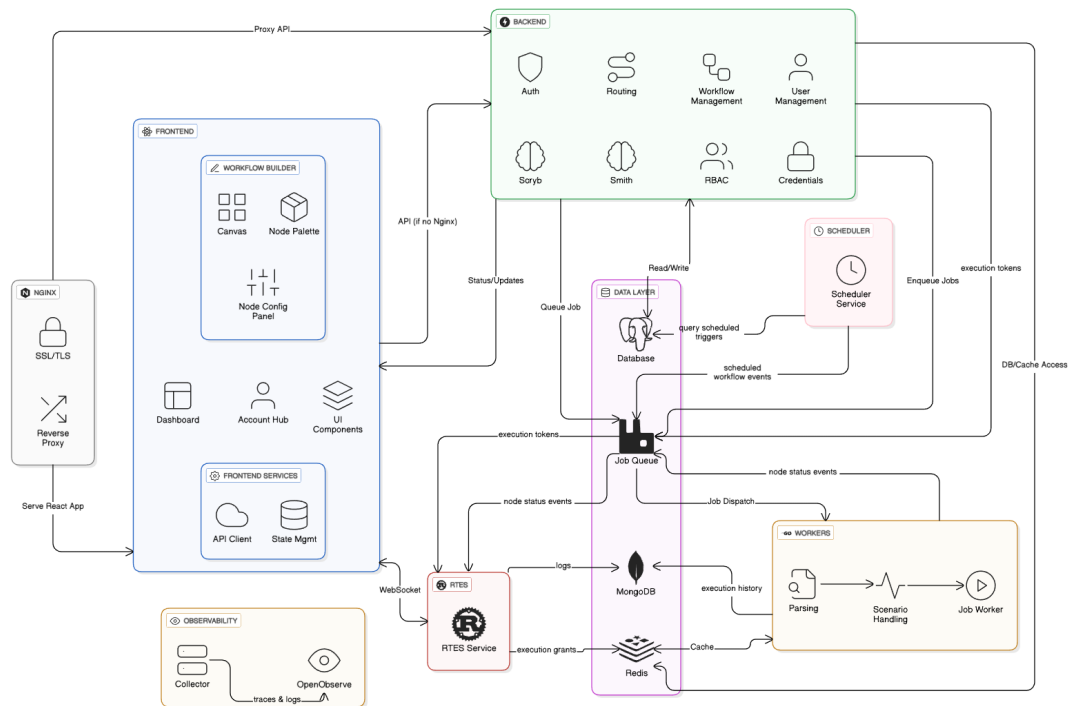


Figure 1: RUNE System Architecture Overview

2.1.1 Components Overview

- **Frontend (Next.js):** User interface and workflow builder
- **Backend API (FastAPI):** Business logic and orchestration
- **Message Broker (RabbitMQ):** Task routing and queuing
- **Worker Engine (Go):** Workflow execution
- **WebSocket Server (Rust):** Real-time updates
- **Database (PostgreSQL):** Data persistence
- **Cache (Redis):** Session and state management

2.2 Technology Stack

Component	Technology	Purpose
Backend API	FastAPI	Business logic
Worker Engine	Go	Workflow execution
WebSocket Server	Rust	Real-time updates
Database	PostgreSQL	Persistent storage
Cache	Redis	State management
Message Queue	RabbitMQ	Task routing
Frontend	Next.js	Web UI

Table 1: Technology Stack Overview

2.2.1 Why FastAPI?

FastAPI provides an async-first, lightweight HTTP surface with excellent developer ergonomics. It delivers high throughput with low latency, automatic OpenAPI generation for client tooling, and strong typing via Pydantic which reduces runtime errors while keeping the control plane responsive.

2.2.2 Why Go?

Go is chosen for the Worker Engine because it delivers predictable, high-throughput concurrency with minimal operational complexity. Goroutines and channel-based patterns make implementing concurrent node execution simple and efficient; Go binaries are compact, have fast startup, and exhibit low tail-latency under load—helpful when we require minimal delays and maximum sustained performance.

2.2.3 Why Rust?

Rust powers the RTES to minimize end-to-end WebSocket latency and maximize message throughput. Rust’s zero-cost abstractions and absence of a garbage collector deliver deterministic performance and tiny runtime overhead, which is ideal for a high-concurrency, low-latency real-time router that must serve many simultaneous connections with minimal jitter.

2.2.4 Why RabbitMQ?

RabbitMQ provides durable queues, exchange-based routing, dead-letter handling, and mature tooling for message acknowledgements and prefetch controls—features that match RUNE’s need for reliable, ordered task delivery and flexible routing topologies.

2.2.5 Why PostgreSQL?

PostgreSQL is used for authoritative persistence because of its ACID guarantees, rich indexing, and JSONB support for schema-evolving workflow definitions. It provides reliable transactional semantics for execution records and audit logs.

2.2.6 Why Redis?

Redis is used for low-latency caching, ephemeral continuation storage for wait/resume semantics, fast pub/sub for inter-service event distribution, and short-lived session/state management—all of which reduce load on the primary database and speed up the execution path.

2.3 Communication Flow

Typical request flow (concise):

1. UI sends execution request to Backend Master; Master validates and creates an execution record.
2. Master constructs a compact `NodeExecutionMessage` (graph, start node, context) and publishes it to RabbitMQ.
3. A Go Worker consumes the message, resolves parameters, runs the node, and publishes lightweight status updates and downstream `NodeExecutionMessages` as needed.
4. Status updates are published to Redis (and optionally to RabbitMQ) so RTES instances can broadcast updates to WebSocket clients in near real time.
5. For wait nodes, the Worker stores continuation state in Redis and the scheduler publishes a resume message when the timer fires.
6. On terminal state, a `CompletionMessage` is published; the Master reconciles results and persists final artifacts in PostgreSQL.
7. Observability: structured logs, traces, and metrics are emitted at each step to correlate and diagnose latency or failures.

2.4 Workflow DSL Structure

RUNE workflows are defined using a JSON-based Domain-Specific Language (DSL) that provides a declarative, structured representation of automation logic. The DSL serves as the canonical format for workflow persistence, execution, and exchange. The workflow definition sent to workers contains only essential execution data, while metadata such as name, description, and version are maintained in the master service.

2.4.1 Core DSL Components

Workflow Definition

A workflow consists of execution metadata and a directed graph of nodes:

- `workflow_id` — Unique workflow identifier
- `execution_id` — Unique identifier for this execution instance
- `nodes` — Array of node definitions
- `edges` — Array of connections between nodes

Note: UI-level metadata (name, description, version) is stored separately in the master service and not included in execution payloads.

Node Definition

Each node represents a discrete execution unit:

```
1 {
2   "id": "node_abc123",
3   "name": "Fetch User Data",
4   "trigger": false,
5   "type": "http",
6   "parameters": {
7     "method": "GET",
8     "url": "https://api.example.com/users/$input.userId",
9     "headers": {"Content-Type": "application/json"}
10  },
11  "credentials": {
12    "id": "cred_xyz",
13    "name": "API Credentials",
14    "type": "api_key",
15    "values": {"key": "..."}
16  },
17  "output": {},
18  "error": {
19    "type": "halt"
20  }
21 }
```

Key node fields:

- `id` — Unique identifier within the workflow
- `name` — Human-readable node name
- `trigger` — Boolean indicating if this node initiates execution

- **type** — Node type identifier (see supported types below)
- **parameters** — Type-specific configuration object
- **credentials** — Optional credential object with encrypted values
- **output** — Placeholder for execution output (empty in definition)
- **error** — Optional error handling configuration

Edge Definition

Edges define control flow between nodes:

```
1 {  
2   "id": "edge_1",  
3   "src": "node_abc123",  
4   "dst": "node_def456"  
5 }
```

Edge fields:

- **id** — Unique identifier for the edge
- **src** — Source node ID
- **dst** — Destination node ID

Note: UI-level fields like labels are maintained separately and not included in execution payloads.

2.4.2 Supported Node Types

RUNE supports twelve node types organized into categories:

Trigger Nodes:

- **ManualTrigger** — Manual workflow initiation

Action Nodes:

- **http** — HTTP requests with retry and timeout support
- **smtp** — Email delivery via SMTP
- **log** — Debug logging with template interpolation
- **agent** — AI agent execution

Control Flow Nodes:

- **conditional** — Binary branching based on expressions
- **switch** — Multi-way branching based on rules
- **wait** — Suspend execution for a specified duration
- **merge** — Synchronize multiple execution paths (barrier/race)

Data Transformation Nodes:

- **edit** — Transform JSON payload with field assignments and expressions
- **split** — Fan-out pattern: split array into parallel execution threads
- **aggregator** — Fan-in pattern: synchronize parallel execution branches

2.4.3 Parameter Resolution

The DSL supports dynamic parameter binding through template expressions evaluated at runtime:

Context References:

- `$node_name` — Access entire output of a previous node
- `$node_name.field` — Access specific field from node output
- `$node_name.body.nested.field` — Access nested fields using dot notation
- `$node_name.array[0]` — Access array elements
- `$input.field` — Access workflow input parameters

Expression Evaluation (Edit Node):

The edit node supports JavaScript expressions in double curly braces:

- `{{ $json.price * 1.2 }}` — Mathematical calculations
- `{{ $json.first_name + ' ' + $json.last_name }}` — String concatenation
- `{{ $node_name.body.status == 'active' }}` — Comparisons

Expressions are evaluated in a sandboxed JavaScript environment (Goja) with access to the accumulated workflow context.

2.4.4 Error Handling

Nodes can specify error handling strategies:

- **halt** — Stop execution and mark workflow as failed
- **ignore** — Continue execution, treating the error as non-fatal
- **branch** — Route execution to an error handling node via `error_edge`

2.4.5 Schema Validation

Workflow definitions are validated against a JSON schema before persistence. The schema enforces:

- Required fields and type constraints for all node types
- Valid node type references (must be one of the twelve supported types)

- Acyclic graph structure (workflows must be directed acyclic graphs)
- Credential reference integrity (credentials must exist and match required types)
- Edge source and destination nodes must exist in the workflow
- At least one trigger node must be present

The DSL design prioritizes clarity, machine readability, and version stability to support workflow portability and tooling integration across the frontend, backend, and worker services.

3 Core Components

This section presents the core components of the RUNE platform, focusing on the frontend application responsible for workflow design, execution monitoring, and user interaction.

3.1 Frontend Application

The frontend application provides the primary interface through which users interact with the RUNE platform. It enables workflow creation, execution, monitoring, and management through a web-based user interface designed for usability and real-time feedback.

3.1.1 User Interface Components

Home Page (Landing Page)

The Home Page serves as the primary entry point for unauthenticated users and provides an overview of the RUNE platform. Its purpose is to communicate the system's value proposition, highlight core features, and guide users toward documentation or application usage.

The homepage is implemented using a dedicated marketing layout that is isolated from the authenticated application interface. This separation ensures a clear distinction between promotional content and operational workflow functionality.

Key elements of the home page include:

- A top navigation bar providing access to documentation, use cases, and product resources.
- A hero section presenting the platform's main value proposition and primary calls to action.
- A feature overview section highlighting core capabilities such as visual workflow design, real-time execution tracing, and role-based access control.
- A step-by-step explanation of how workflows are created and executed within the platform.
- An informational section describing the platform's mission and intended use cases.

Authentication-aware navigation ensures that returning users are redirected to the dashboard, while new users are encouraged to explore the system's capabilities before registration.

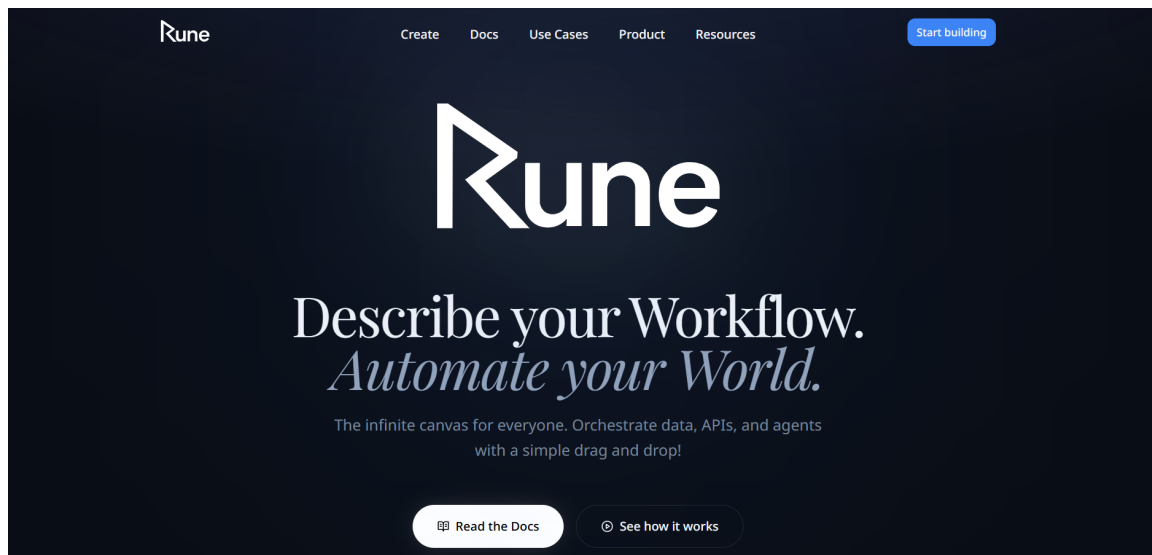


Figure 2: Home Page

Dashboard

The dashboard acts as the central entry point of the system. It provides users with an overview of their workflows, execution activity, system-level statistics, and comprehensive management capabilities.

Key functionalities include:

- **Workflow Management:** Displaying a list of workflows accessible to the user along with their current status, creation options, and quick access to workflow templates.
- **Execution Monitoring:** Presenting execution history with success, failure, and running states, along with detailed execution logs and real-time status updates.
- **Analytics & Statistics:** Showing aggregated statistics such as execution counts, recent activity trends, and performance metrics across workflows.
- **Credentials & Integrations:** Managing authentication credentials and API keys for external service integrations (HTTP endpoints, SMTP servers, webhooks, etc.), with secure storage and per-user access control.
- **Template Library:** Browsing and deploying pre-configured workflow templates for common automation patterns, enabling rapid workflow creation and best-practice implementations.
- **Profile Management:** User profile management including authentication settings and account configuration options accessible through the navigation interface.

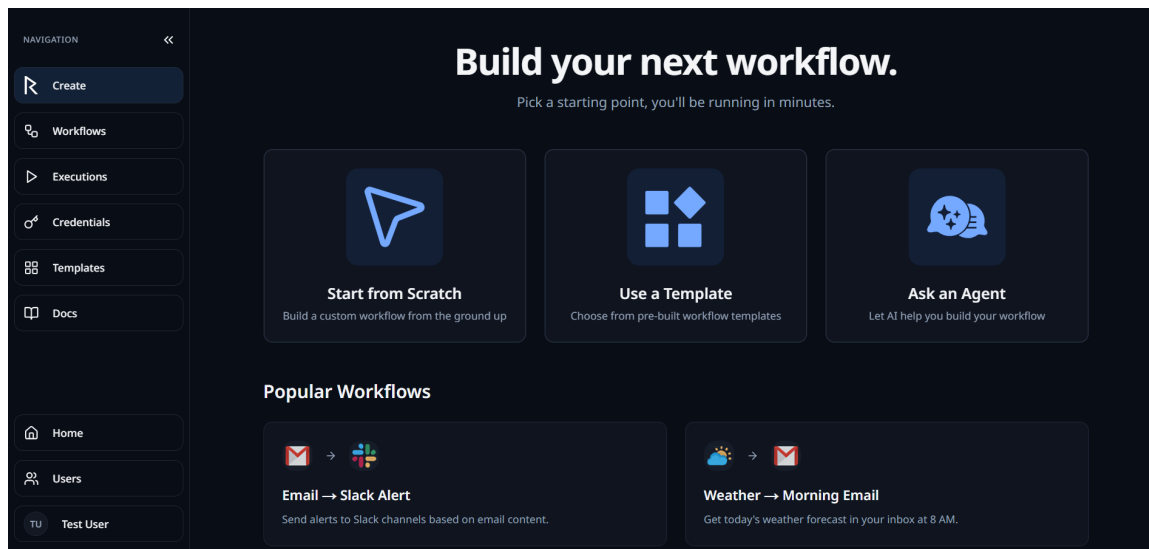


Figure 3: Dashboard page

Canvas Workflow Builder

The Canvas Workflow Builder is the core interface for designing workflows. It provides a visual, node-based editor that allows users to construct workflows using a drag-and-drop interaction model with comprehensive editing and management capabilities.

Main features include:

- **Node Libraries:** Multiple categorized node palettes containing available workflow components, including core nodes (triggers, actions, conditionals), integration nodes (HTTP requests, SMTP, webhooks), control flow nodes (switches, merges, wait), and data transformation nodes (edit, split, aggregate).
- **Visual Workflow Design:** Drag-and-drop placement of nodes onto the canvas with visual connections representing data and execution flow between nodes with real-time validation and error highlighting.
- **Action Bar Controls:** A comprehensive toolbar providing workflow management operations including:
 - Workflow execution controls (run, stop)
 - Edit operations (undo, redo, copy, paste)
 - Version history access and workflow snapshot navigation
 - Import/export workflows
 - Save and publish workflow operations
 - View mode toggles and canvas zoom controls

- **Node Inspector:** A panel to display and allow modification of node-specific configurations, including:
 - Parameter configuration
 - Credential and integration settings
 - Conditional logic and expression builders
 - Real-time input/output view
- **AI-Assisted Workflow Generation:** Integrated AI capabilities for intelligent workflow construction (Smith) and natural language workflow description generation (Scryb).

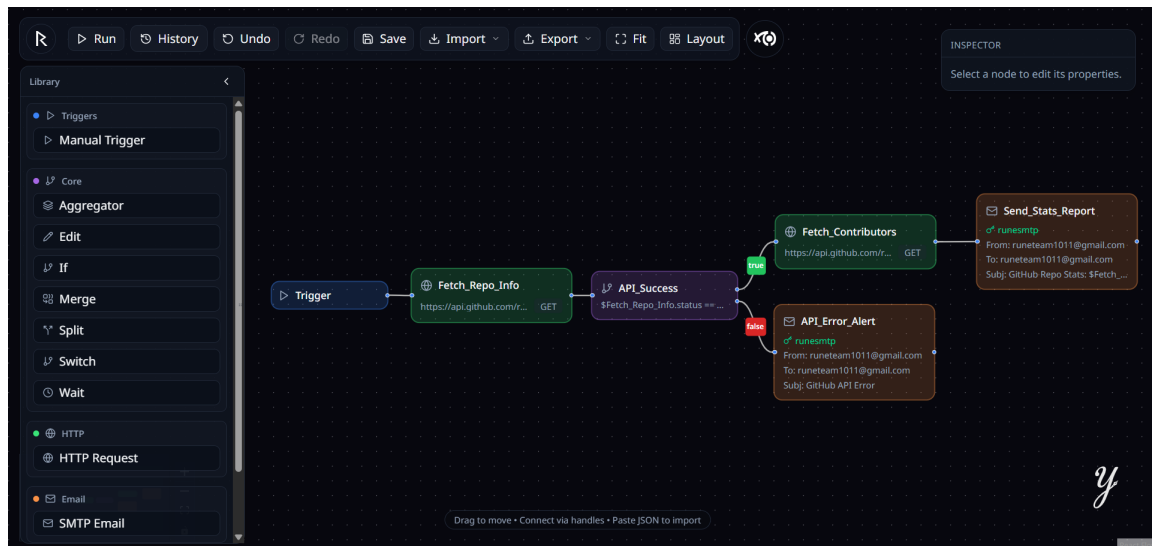


Figure 4: Canvas Page

Real-time Execution Viewer

The real-time execution viewer enables live monitoring of workflow executions directly within the canvas. During execution, visual feedback is provided to reflect the current state of each node.

Key capabilities include:

- Live node status updates (running, completed, failed).
- Display of execution logs and error messages.
- Progress indicators showing overall execution state.

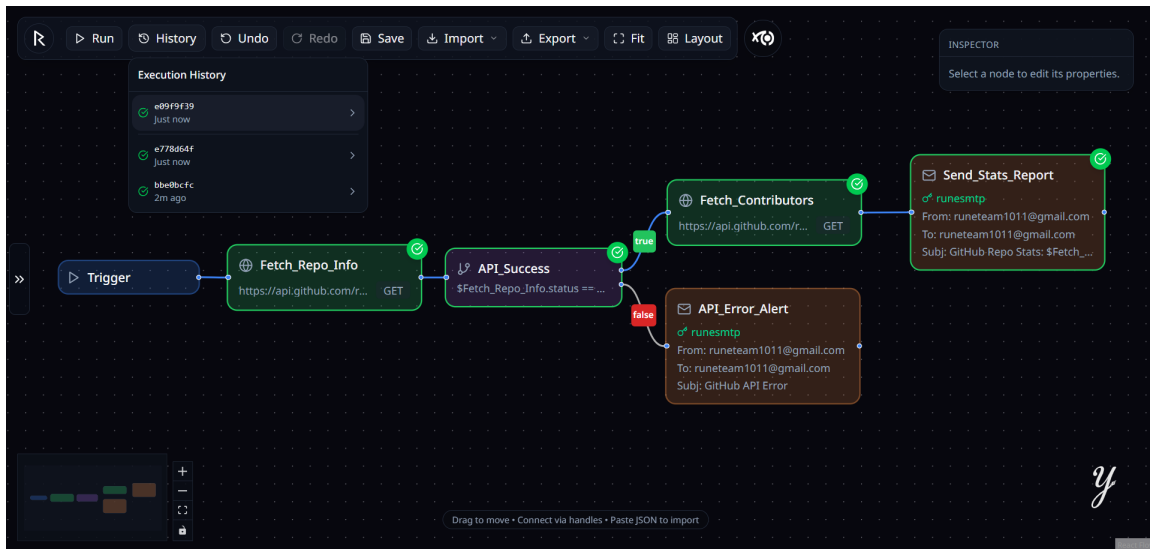


Figure 5: Real-time Workflow Execution

3.1.2 WebSocket Client Integration

The RUNE frontend integrates a WebSocket client to enable real-time monitoring of workflow execution. When a workflow execution is initiated, the backend generates a unique execution identifier along with a temporary access token. The frontend establishes a WebSocket connection to the Rune token execution service using this identifier, allowing continuous streaming of execution updates.

The WebSocket client handles key lifecycle events, including connection establishment, message reception, error handling, and disconnection. Incoming messages represent execution state changes at both workflow and node levels and are immediately reflected in the visual workflow canvas. Upon successful connection, the server replays previously executed node states to ensure state consistency even if the client connects after execution has already started.

To ensure reliability, the client implements an automatic reconnection strategy based on exponential backoff. In the event of an unexpected disconnection, the client attempts to reconnect within predefined limits, improving resilience against transient network failures while preventing excessive retry attempts.

3.1.3 State Management

Global state management in the RUNE frontend is implemented using the React Context API combined with the `useReducer` hook. This approach follows a unidirectional data flow pattern, centralizing shared application state such as authenticated user information, accessible workflows, loading indicators, and global error states.

State updates are performed through explicitly defined actions dispatched to a reducer,

ensuring predictable and traceable state transitions. Asynchronous actions are responsible for initializing application data and refreshing workflows, including permission resolution for each workflow. This design promotes loose coupling between components, improves maintainability, and enables consistent state access across the application without relying on external state management libraries.

3.2 Backend Master (Python - FastAPI)

3.2.1 Overview

The Backend Master is the platform's central orchestrator. Implemented with FastAPI, it manages client-facing APIs, validates and persists workflow definitions, coordinates executions with the worker fleet, and aggregates execution results and logs. It is responsible for enforcing access control, handling secrets, and exposing observability endpoints for monitoring and health checks.

3.2.2 Design and Responsibilities

Built for asynchronous request handling and clear operational boundaries, the service has three primary responsibilities: first, accept and validate workflow definitions and user commands; second, persist authoritative state in PostgreSQL and handle secrets securely; third, prepare and dispatch execution payloads to RabbitMQ while tracking execution lifecycle and results. Real-time user updates are propagated via the WebSocket subsystem using Redis to decouple broadcast concerns from core request handling.

3.2.3 Key Capabilities

Validation is performed using Pydantic models and JSON schema checks before any state is committed. Workflow definitions are stored as JSONB documents to allow schema evolution while maintaining relational integrity for metadata and permissions. The execution coordinator enriches payloads with credentials (retrieved and decrypted on-demand), enforces execution policies (timeouts, concurrency limits, retry rules), and publishes messages to the appropriate exchange/queue.

Authentication uses JWT for stateless sessions and a role-based access control model for per-workflow permissions. Sensitive values are encrypted at rest with AES-256 and rotated according to a key-management policy.

3.2.4 Workflow Lifecycle (high level)

Saving a workflow: the client submits the definition, the Master validates schema and trigger type, checks the caller's role and permissions, stores a versioned snapshot in Post-

greSQL, records credential references (encrypted), and returns the snapshot id.

Running a workflow: the client requests a run for a snapshot, the Master creates a pending execution record, resolves inputs and required credentials, publishes a compact execution payload to the `runs` queue, and returns an `execution_id`. Workers execute nodes, emit short status updates via Redis/`status`, and publish a `CompletionMessage`; the Master reconciles completion and persists final results.

Worker instances consume execution messages from the queue, perform node work, and publish lightweight status events and completion messages via Redis and RabbitMQ so other services can observe progress. The RTES (RUNE Token Execution Service) is responsible for handling WebSocket client connections and live broadcasting: the Backend Master issues a short-lived access token. The frontend requests this token from the Master and uses it to authenticate directly with RTES.

3.2.5 Resilience and Operational Considerations

To ensure reliability the service enforces idempotency keys for externally triggered operations, uses transactional updates to avoid partial state, and exposes retry semantics for transient failures. Long-running requests are handled asynchronously via the message bus to keep the HTTP surface fast and reliable. For scaling, stateless API instances can be increased behind a load balancer while shared state remains in PostgreSQL, Redis, and RabbitMQ.

3.2.6 Extensibility

The backend is designed as a set of modular services: validation and persistence, execution coordination, secrets management, and telemetry. This separation allows features such as new trigger types, alternate credential backends, or support for additional worker runtimes to be introduced with minimal changes to the core orchestrator.

3.3 Queues (RabbitMQ)

RUNE uses four RabbitMQ queues to coordinate work between workers and the backend. All queues connect to the central `rune.exchange` and are configured for reliability and appropriate message delivery guarantees.

3.3.1 Global exchange and conventions

All queues bind to the direct exchange `rune.exchange` with individual routing keys. Failed messages are automatically routed to dead-letter queues for later investigation and recovery.

3.3.2 runs

The main work queue that distributes execution tasks to workers. Messages are durable and persistent to ensure no work is lost if a worker crashes. Workers process one execution at a time and manually confirm completion before handling the next task. Each message contains the complete execution specification needed to run the workflow.

3.3.3 status

A fast, lightweight queue for continuous progress updates during execution. Messages are non-persistent to prioritize throughput; they expire after 30 seconds since they represent real-time progress that becomes stale quickly. The backend and status service consume these updates to track execution progress and collect logs without needing long-term storage.

3.3.4 completion

The final-results queue that signals when an execution finishes. Messages are durable and persistent, containing the execution outcome, results, and metadata. The backend consumes these messages, saves the final results to permanent storage, and triggers any post-execution workflows or cleanup tasks.

3.3.5 token

A secure, short-lived queue for distributing credentials and access tokens to workers. Messages expire after 60 seconds and are encrypted at the application layer. Workers must validate the sender's identity before using any tokens received from this queue .

3.4 RUNE Worker Engine (Go)

3.4.1 Overview

The Worker Engine is the core execution runtime of the RUNE platform, responsible for consuming workflow execution requests, orchestrating node execution, and publishing results. Implemented in Go, it is designed to be stateless, horizontally scalable, and fault-tolerant.

3.4.2 Architecture and Design

The worker follows a message-driven architecture with clear separation of concerns. It consists of multiple consumers that handle different message types, an executor that coordinates node execution, and a registry of pluggable node implementations. All components

operate asynchronously using Go's concurrency primitives, ensuring efficient resource utilization and responsive execution.

3.4.3 Core Components

Message Consumers

Two primary consumers handle incoming workflow operations. The Workflow Consumer processes execution requests from the **runs** queue, validating payloads and delegating to the executor. The Resume Consumer handles continuation messages for suspended workflows, particularly those paused by wait nodes.

Execution Engine

The executor orchestrates the complete node execution lifecycle. It constructs the execution context, resolves dynamic parameters from accumulated workflow state, invokes registered node implementations, applies configured error strategies, and publishes status updates and completion messages. The executor implements recursive execution by publishing downstream node messages, enabling distributed workflow traversal.

Node Registry

A pluggable registry maintains factories for all available node types. Current implementations include HTTP requests, SMTP email delivery, conditional branching, data transformation (split, merge, edit), flow control (switch, wait), and aggregation. New nodes in the future can be added by implementing the standard interface and registering during initialization.

Parameter Resolution

The resolver component processes dynamic parameter references before node execution. It replaces placeholder expressions with actual values from the accumulated execution context, supporting field references and nested data access.

Scheduler Integration

A Redis-backed scheduler manages deferred execution for wait nodes. When a wait node is encountered, the executor stores the continuation payload and schedules a resume event. The scheduler publishes resume messages when timers expire, allowing workflows to continue seamlessly.

3.4.4 Key Capabilities

- **Stateless Execution:** Workers maintain no persistent state, enabling horizontal scaling and fault tolerance through message redelivery
- **Dynamic Parameter Binding:** Runtime resolution of parameter references ensures nodes receive concrete, context-aware inputs

- **Error Handling Strategies:** Per-node configuration of failure behavior including halt, ignore, and conditional branching
- **Control Flow Support:** Native implementation of branching, loops, parallel execution, and synchronization primitives
- **Extensible Node System:** New capabilities added through self-registering node implementations without core modification
- **Observable Execution:** Structured status messages enable real-time monitoring and detailed execution logs

3.4.5 Execution Flow

When a workflow execution message arrives, the consumer validates the payload and constructs an execution context containing the node graph, accumulated state, and credentials. The executor identifies the current node, resolves its parameters against the context, instantiates the corresponding node implementation, and invokes its execution method.

Upon successful completion, the executor updates the accumulated context with node outputs, determines downstream nodes based on the workflow graph and any conditional logic, and publishes execution messages for each successor. Status updates are continuously published to enable real-time UI updates. If execution fails, the configured error strategy determines whether to halt the workflow, skip the node, or route to an error handling branch.

For wait nodes, execution is suspended by storing the current state and scheduling a resume event. When the timer expires, the scheduler publishes a resume message that re-enters the workflow at the suspension point with full context restoration.

3.4.6 Operational Characteristics

Reliability: RabbitMQ integration provides durable message delivery with manual acknowledgment. Failed executions are requeued automatically, ensuring no work is lost during worker failures.

Scalability: Multiple worker instances consume from shared queues with configurable prefetch limits. Processing capacity scales linearly with worker count.

Resource Management: Context-based cancellation ensures proper cleanup of goroutines and network connections. Connection pooling reduces overhead for external service calls.

Security: Credentials are received encrypted and decrypted only during node execution. No plaintext secrets are logged or persisted by workers.

3.4.7 Message Protocols

Workers interact with the platform through well-defined message schemas. Execution messages contain the complete node definition, resolved credentials, and accumulated context. Status messages provide incremental updates on execution progress. Completion messages contain final results, generated artifacts, execution duration, and terminal status.

3.4.8 Implementation Structure

The codebase is organized into distinct packages for messaging (`pkg/messaging`), execution logic (`pkg/executor`), node implementations (`pkg/nodes/custom`), parameter resolution (`pkg/resolver`), scheduling (`pkg/scheduler`), and platform abstractions (`pkg/platform`). This modular structure facilitates testing, maintenance, and future enhancements.

3.5 RUNE Token Execution Service (RTES) - WebSocket Server (Rust)

3.5.1 Overview

The RUNE Token Execution Service (RTES) is a specialized WebSocket server implemented in Rust that provides live, bidirectional communication between the backend and frontend during workflow execution. It enables users to monitor workflow progress in real time, receiving immediate updates on node completion, status changes, and execution results without requiring page refreshes or polling.

3.5.2 Purpose and Responsibilities

RTES serves as the communication bridge for execution monitoring and control. Its primary responsibilities include:

- **Real-Time Status Streaming:** Broadcasting execution status updates from the Backend Master and Worker Engine to connected frontend clients.
- **Client Connection Management:** Establishing, maintaining, and gracefully closing WebSocket connections with frontend clients.
- **Message Routing and Distribution:** Receiving events from the message queue and the Backend Master via Redis, then routing them to appropriate clients based on execution context.
- **State Consistency:** Ensuring that clients receive complete execution history, including nodes executed before connection establishment, to maintain visual consistency in the workflow canvas.

- **Error Handling and Recovery:** Implementing automatic reconnection strategies on the client side to handle transient network failures gracefully.

3.5.3 Architecture and Design

The RTES is designed with a distributed, event-driven architecture. Rather than maintaining direct connections to all clients, it leverages Redis Pub/Sub to receive execution events published by the Backend Master and Worker Engine. This decoupling allows the RTES to scale horizontally—multiple RTES instances can be deployed behind a load balancer, and all receive events through the shared Redis channel.

The service maintains an in-memory mapping of active client connections, organized by execution identifier. When an event arrives via Redis, the service identifies the relevant clients and delivers the message over their WebSocket connections.

3.5.4 Key Capabilities

- **High-Performance Message Delivery:** Efficient handling of high-frequency status updates from multiple concurrent workflow executions.
- **Stateless Design:** No persistent storage of execution data, allowing instances to be added or removed without state coordination.
- **Connection Resilience:** Automatic client reconnection with state recovery ensures users receive continuous updates even if their network is temporarily interrupted.
- **Scalable Architecture:** Pub/Sub-based event distribution allows the service to support thousands of concurrent clients across multiple instances.
- **Low-Latency Updates:** Direct WebSocket connections minimize update latency, providing near-real-time visual feedback in the user interface.

3.5.5 Integration with Backend Systems

The RTES integrates seamlessly with the Backend Master and Worker Engine through the message queue and Redis. When a workflow is initiated, the Backend Master obtains a temporary access token for the client and establishes the execution context. The client uses this token to authenticate the WebSocket connection to the RTES. As workers execute nodes and publish status updates, these messages flow through the system and are broadcast by the RTES to all clients monitoring that execution.

3.5.6 User Experience Benefits

By providing real-time updates, the RTES eliminates the need for users to manually refresh pages or wait for background polling intervals. Users observe workflow progress as it happens, with node states updating visually on the canvas. This immediate feedback enhances the user experience and builds confidence in the platform's reliability and responsiveness.

3.6 Redis Cache Layer

3.6.1 Overview

Redis serves as the distributed caching and state management layer for the RUNE platform, providing fast, reliable data access across multiple system components. It plays a critical role in maintaining session state, caching transient data, and enabling real-time communication between backend services and frontend clients.

3.6.2 Role Across System Components

The Redis cache layer is integrated into multiple layers of the RUNE architecture:

Backend Master Integration

The Backend Master uses Redis to cache frequently accessed data such as workflow metadata, user permissions, and execution state. This reduces database query overhead and improves API response times. Session tokens and user authentication data are stored with appropriate time-to-live (TTL) values to balance security and performance.

Worker Engine Integration

Workers utilize Redis for storing intermediate execution state and lineage information during workflow execution. When workflows are paused (e.g., by wait nodes), continuation payloads are retained in Redis with configured expiration periods. This enables workers to maintain execution context without relying on long-term database writes, improving execution throughput.

Real-Time Execution Service (RTES) Integration

The RTES leverages Redis's Pub/Sub mechanism to broadcast execution updates across multiple server instances. When execution status changes occur, the RTES publishes messages to Redis channels, which are then distributed to all connected WebSocket clients. This decoupled architecture allows the RTES to scale horizontally while maintaining real-time communication consistency.

3.6.3 Key Functions

- **Session and User State Management:** Storage of authenticated user sessions with configurable expiration periods.
- **Execution Context Caching:** Temporary storage of workflow execution state and accumulated context during node execution.
- **Distributed Pub/Sub:** Real-time event broadcasting for execution status updates and cross-service communication.
- **Workflow Metadata Caching:** Caching of workflow definitions and permission metadata to reduce database load.
- **Rate Limiting:** Tracking request rates to enforce usage limits and prevent abuse.
- **Timer Management:** Storage of pending timers for deferred workflow execution (e.g., wait node resumption).

3.6.4 Reliability and Data Consistency

Redis is configured for high availability through replication and persistence mechanisms. Critical execution state is replicated across multiple Redis instances to ensure continuity during failures. Data with high business value is persisted to disk to survive system restarts, while ephemeral data such as session tokens may be stored only in memory with appropriate TTL values.

3.6.5 Performance Benefits

By caching frequently accessed data and reducing reliance on database queries, Redis significantly improves system performance. The Pub/Sub mechanism enables real-time updates without polling or excessive database queries, allowing the frontend to display execution progress with minimal latency. This contributes to a responsive user experience during workflow execution and monitoring.

3.7 AI Integration Components

3.7.1 Overview

The RUNE platform incorporates artificial intelligence capabilities to enhance user productivity and reduce the friction associated with workflow creation and documentation. Two primary AI-driven components—**Smith** and **Scryb**—represent the platform’s commitment to intelligent automation and accessibility.

3.7.2 Smith - Workflow Generator

Purpose

Smith is an AI-powered workflow generation system that translates natural language descriptions into executable workflow definitions. This component significantly lowers the barrier to entry for workflow creation, allowing users without deep technical knowledge to design sophisticated automation workflows through simple natural language prompts.

Key Capabilities

- **Natural Language Processing:** Interprets user-provided workflow descriptions in plain language, extracting intent, required actions, and control flow logic.
- **Intelligent Node Recommendation:** Analyzes the workflow intent and automatically suggests appropriate node types from the available registry (HTTP calls, email delivery, data transformation, conditional branching, etc.).
- **Workflow Structure Generation:** Constructs a complete workflow definition including node connections, parameters, and error handling strategies based on the interpreted intent.
- **Validation and Refinement:** Performs structural validation to ensure the generated workflow is syntactically correct and semantically sound before presenting it to the user.
- **Interactive Refinement:** Supports iterative refinement where users can provide feedback on generated workflows, allowing Smith to improve subsequent generations.

Value Proposition

Smith democratizes workflow creation by eliminating the need for users to manually construct node graphs. Instead of clicking and dragging nodes, users simply describe their automation goals. The system handles the complexity of translating those goals into technical workflow structures. This capability is particularly valuable for organizations seeking to rapidly prototype and deploy automation solutions.

3.7.3 Scryb - Documentation Generator

Purpose

Scryb is an automated documentation system that generates comprehensive, human-readable documentation for workflows. It ensures that all created workflows are thoroughly documented, improving maintainability, knowledge transfer, and organizational compliance.

Key Capabilities

- **Automatic Markdown Generation:** Analyzes workflow structure and generates well-formatted Markdown documentation without manual effort.

- **Semantic Understanding:** Infers workflow purpose, data flows, and control logic from the workflow definition and parameters, translating technical structure into business-friendly language.
- **Comprehensive Documentation:** Generates sections including workflow overview, node descriptions, parameter explanations, error handling procedures, and execution examples.
- **Consistency and Standardization:** Ensures all generated documentation follows consistent formatting and structure, improving readability and professionalism.
- **Integration with Workflow Lifecycle:** Supports documentation regeneration when workflows are modified, keeping documentation synchronized with implementation.

Value Proposition

Scryb addresses a common challenge in automation projects: the tendency for documentation to become outdated as workflows evolve. By automatically generating documentation from the authoritative workflow definition, Scryb ensures that documentation is always current and accurate. This reduces maintenance overhead and supports knowledge sharing across teams.

3.7.4 Integration with RUNE Platform

Both Smith and Scryb integrate directly into the user workflow. Smith is available during the workflow creation phase, allowing users to jumpstart development. Scryb is accessible after workflow creation, enabling users to generate professional documentation for sharing with stakeholders or archiving for compliance purposes.

These AI components represent the platform's vision of making workflow automation accessible to users of varying technical backgrounds while maintaining high standards for documentation and maintainability.

4 Data Flow & Execution Lifecycle

4.1 Overview

This section describes the end-to-end execution lifecycle of workflows in RUNE, reflecting the implemented components and their interactions. The description follows the observed runtime behaviour: the Backend Master prepares and publishes execution payloads, the Worker Engine consumes and executes nodes, Redis and the scheduler manage deferred execution, and the RUNE Token Execution Service (RTES) streams status updates to connected clients.

4.2 High-level Execution Flow

The following sequence summarizes the executed path for a typical workflow run. Each step maps to implemented code paths and components present in the repository.

1. **Trigger and request validation:** A user triggers execution from the frontend (UI). The Backend Master validates permissions, resolves credentials, and creates an authoritative execution record in the persistent data store.
2. **Payload construction:** The Backend Master flattens and enriches the workflow definition with resolved credentials and input parameters. The service constructs a well-formed execution message that contains the workflow graph, the starting node identifier, and the initial accumulated context.
3. **Publishing to the message bus:** The execution payload is published to the platform message bus (the configured exchange/queue). This hands off responsibility to the Worker Engine and decouples synchronous API responses from long-running execution.
4. **Worker consumption and orchestration:** A Worker Engine instance consumes the `NodeExecutionMessage`. The consumer validates and forwards the payload to the Executor component, which builds an execution context and resolves any parameter references against the accumulated context.
5. **Node invocation:** The Executor instantiates the target node implementation from the node registry and invokes its "Execute" method within the provided "ExecutionContext". Node implementations produce output that the executor uses to update the accumulated context.

6. **Control-flow handling:** After node completion the executor interprets control-flow semantics (split, merge, conditional, switch, aggregator). For fan-out operations, the executor publishes new `NodeExecutionMessages` for downstream nodes; for terminal states, the executor publishes a `CompletionMessage`.
7. **Deferred execution (wait nodes):** When a node intentionally suspends execution (for example, a `wait` node), the Executor records the continuation payload and scheduling metadata. The scheduler component (backed by Redis) will later publish a resume message that re-enters the workflow at the suspension point.
8. **Status and completion propagation:** Throughout execution the worker publishes concise node status updates. These status messages are consumed by services that persist execution logs, update the execution record, and broadcast live updates via the RTES to connected frontend clients.
9. **Finalization:** When the workflow reaches a terminal state (completed, failed, or halted), the executor publishes a `CompletionMessage`. The Backend Master reconciles the completion, persists final results and artifacts as needed, and marks the execution record accordingly.

4.3 Message and Data Lifecycles

This subsection explains how messages and transient data move through the system during execution.

NodeExecutionMessage: Represents a single node execution request. Initially published by the Backend Master and subsequently published by workers for downstream nodes. The message carries the workflow definition, current node id, accumulated context, and lineage information for split branches.

NodeStatusMessage: Short-lived status updates (running, success, failed, waiting) emitted by workers during node execution. Status messages are used for real-time UI updates and for lightweight telemetry; they are not relied upon for long-term persistence.

CompletionMessage: Published once per execution when a terminal state is reached. Completion messages include final context, status, execution duration, and artifact references. The Backend Master consumes completion messages to finalize execution records.

4.4 Wait / Resume Semantics

RUNE supports deferred execution through dedicated wait/resume semantics:

- When a node intentionally pauses (e.g., `wait`), the Executor publishes a waiting status and stores the continuation payload where the scheduler can access it (Redis).

- The Scheduler service monitors timers and scheduled triggers; when a timer expires it publishes a resume message to the resume queue.
- The Resume Consumer rehydrates the continuation context and republishes Node-ExecutionMessages for the downstream nodes, allowing execution to continue from the suspension point.

4.5 Error Handling and Retry Behavior

Error handling is implemented at node and executor levels. Nodes may declare an error policy (halt, ignore, branch) that the Executor enforces:

- **Halt:** Execution halts and a CompletionMessage with halted status is published.
- **Ignore:** The Executor treats the failure as non-fatal and continues with configured downstream nodes.
- **Branch:** If an error-edge is specified, execution is routed to the defined error handling node.

Transient failures originating from infrastructure (message delivery, network blips) rely on message requeueing and consumer retry semantics. Persistent failures are surfaced through status messages and stored in execution logs for diagnosis.

4.6 Real-time Updates and RTES Integration

The RTES (RUNE Token Execution Service) subscribes to status events and broadcasts them to frontend clients over WebSocket. The design guarantees that:

- Clients joining late receive a replay of relevant node statuses to reconstruct the current execution view.
- Status updates are lightweight, enabling low-latency UI updates even during high-concurrency runs.

Redis is used as the cross-service message distribution mechanism between Backend Master, Worker Engine, scheduler, and RTES to ensure consistent event propagation in multi-instance deployments.

4.7 Observability and Auditing

Throughout the execution lifecycle the platform produces structured logs, execution traces, and metrics:

- **Structured Logs:** Workers and the Backend Master emit structured JSON logs for each node execution and lifecycle event.
- **Execution Traces:** Tracing context is propagated across service boundaries to correlate frontend requests with worker actions.
- **Audit Trail:** The Backend Master persists execution metadata and results to the authoritative data store for auditing and historical analysis.

These artifacts enable post-mortem analysis, performance tuning, and provide a reliable audit trail for compliance requirements.

5 Security

The following summarizes the concrete, implemented security behaviour found in the repository. This section intentionally documents only mechanisms implemented in code and operational config.

5.1 Authentication and Authorization

Implemented items:

- Stateless authentication using signed tokens (API issues short-lived access tokens for clients).
- Role-based access control (RBAC) and per-workflow permissions are implemented; workflows have ‘OWNER’, ‘EDITOR’, and ‘VIEWER’ roles enforced by the backend permission checks.

References: backend permission enforcement and sharing APIs are implemented in the permission service and workflow service code.

5.2 Secrets and Credentials Handling

The codebase implements application-level credential encryption and on-demand resolution. Key points:

- Credentials are stored encrypted in the database.
- The Backend Master performs decryption on demand and embeds resolved credential values into the execution payload sent to workers.
- Short-lived, ephemeral deliveries of credentials/tokens are used for runtime operations (the system publishes time-limited Token messages to a dedicated channel/queue).

5.3 Workflow Execution and Runtime Protections

Implemented runtime behaviour:

- Master–Worker execution: the Backend Master publishes compact execution payloads to RabbitMQ; stateless workers consume tasks and publish status updates.
- Workers receive resolved credentials only at execution time and avoid logging or persisting plaintext secret values.

- Deferred execution (wait nodes) and scheduling use Redis for continuation storage and a scheduler service to publish resume messages.
- Real-time execution updates are delivered via a dedicated execution service (RTES) that uses short-lived access tokens for WebSocket connections.

5.4 Sharing Strategies

Implemented sharing model:

- Workflows are shared by creating ‘WorkflowUser’ permission records that assign ‘EDITOR’ or ‘VIEWER’ roles; sharing and revocation APIs are implemented in the permissions service.
- Owner role is protected (cannot be granted via the sharing API and cannot be revoked via the sharing API).

5.5 Credential Sharing and Workflow Export

The codebase implements credential-level access controls and export hygiene to prevent accidental disclosure of secrets.

- **Credential sharing:** Credentials and workflow access are governed by backend permission checks. Sharing is recorded through permission entities and enforced at resolution time so that credentials are only decrypted and provided for executions authorized by the permission model.
- **Workflow export:** Exported workflow artifacts intentionally exclude plaintext credential values. Exports contain credential references or placeholders and a schema stamp; importing an exported workflow requires explicit re-binding of secrets by an authorized user.

6 Implementation Challenges & Solutions

6.1 Secrets Management and Exporting

Challenge

Protecting sensitive credentials while enabling runtime use and safe export: Workflows and nodes require credentials (API keys, tokens, passwords) for third-party integrations. The challenge is to make these credentials available to runtime components when needed without exposing them in storage, logs, or exported workflow artifacts.

Solution

We implemented a defense-in-depth approach that keeps secrets encrypted at rest, minimizes their exposure in transit, and prevents accidental leakage during exports:

- **Encrypted storage:** Credentials are encrypted in the Backend Master using strong application-level encryption before persisting to the database.
- **On-demand decryption:** Secrets are decrypted only when an execution is initiated and are attached to the execution payload in a controlled, read-only form ("ExecutionContext").
- **Ephemeral delivery:** Short-lived tokens/leases are used for sensitive operations and delivered via dedicated, time-limited channels so that workers receive only what they need for the current execution.
- **No persistent logging or export:** Workflow export mechanisms and logging explicitly exclude credential values. The UI and export formats replace secrets with placeholders and references to credential identifiers rather than plaintext values.
- **Key management and rotation:** Operational procedures are used to rotate encryption keys and retire long-lived credentials as part of deployment practices.

This strategy balances security with usability: workers receive the credentials necessary for execution but never persist them or expose them in exported artifacts.

6.2 Wait Nodes and Scheduling Triggers (Scalable Timers)

Challenge

Supporting deferred execution without central bottlenecks: Workflows often require pauses or scheduled resumptions (for example, waiting for external events or delays). Implementing timers and resume logic must not introduce a single point of failure or prevent horizontal scaling.

Solution

We adopted a distributed scheduler pattern that preserves the platform's scalability and decouples timing concerns from synchronous execution paths:

- **Continuation storage:** When a `wait` node suspends execution, the Executor stores the continuation payload and scheduling metadata in Redis, minimizing database writes and keeping the resume data accessible to the distributed scheduler.
- **Scheduler-as-a-service:** A scheduler component monitors timers and scheduled triggers, and when a trigger fires it publishes a resume message to the resume queue. The resume queue is consumed by the Resume Consumer which re-enters the workflow at the correct node.
- **Decoupled resume path:** Using a dedicated resume queue and resume consumer isolates deferred execution from active workers. This keeps normal execution fast and allows the scheduler and resume handlers to scale independently.
- **Fault tolerance:** Redis and the message bus provide resiliency—scheduled events are stored and retried where appropriate, and resume messages are durable so work is not lost during failures.

This design preserves the system's distributed nature and avoids introducing a central timing authority that would limit horizontal scaling.

6.3 Schema and Configuration Consistency

Challenge

Maintaining a single source of truth across Frontend, Backend Master and Workers: The platform requires consistent workflow definitions and message schemas across three main tiers (UI, Master, Workers). Divergence introduces runtime errors, incorrect execution, and confusing user experiences.

Solution

We established clear contracts and procedures to maintain consistency while allowing independent evolution:

- **Backend as authoritative source:** The Backend Master remains the single source of truth for workflow definitions. The frontend fetches and displays the authoritative definition, while the master publishes the canonical payloads to the message bus for workers.
- **Schema validation and versioning:** Workflow definitions and message schemas are validated against explicit JSON schemas before persistence or publishing. Schema versioning is used to manage backward-incompatible changes, and migration paths are documented for upgrades.
- **Message contracts:** All worker-facing message formats are explicitly defined and implemented in shared message types (the project includes message types used by master and worker code). This reduces ambiguity and enforces compile-time or test-time agreement where possible.
- **Export hygiene:** Exported workflow artifacts intentionally exclude sensitive fields and include a schema version stamp so imported workflows can be validated on ingestion.

Together, these measures enforce a single point of truth while providing safe upgrade paths and predictable runtime behaviour.

6.4 RTES: Execution Logs and Live Updates

Challenge

Delivering high-volume live updates and preserving execution history: Real-time status updates are essential for the UI, but high-frequency events and late-joining clients complicate delivery, ordering, and replay of execution history.

Solution

The RTES and surrounding subsystems address these concerns through separation of concerns and hybrid ephemeral/persistent strategies:

- **Event partitioning by execution:** Status events are partitioned by execution identifier so updates for different workflows do not interfere and can be routed efficiently to interested clients.
- **Lightweight status messages:** Workers publish concise node status messages for live streaming; heavier payloads and artifacts are persisted by the Backend Master and referenced by completion messages to avoid flooding the real-time channel.
- **Replay capability:** RTES ensures late-joining clients receive a short history replay (recent node statuses) so the client can reconstruct the current view without querying the database directly. The authoritative, long-term execution log is stored by the Backend Master for auditing and deep inspection.
- **Scalable delivery:** RTES instances subscribe to Redis Pub/Sub (or equivalent message distribution) to receive events and broadcast to connected WebSocket clients. This enables horizontal scaling of RTES without centralized connection bottlenecks.
- **Ordering and idempotency:** Messages carry sequence or timestamp metadata that enables clients and RTES to order events deterministically and ignore duplicates where necessary.

This approach delivers low-latency live updates while keeping the persistence layer authoritative for history and analysis.

6.5 Additional Operational Challenges

Challenge

High-throughput and back-pressure management: Handling bursts of executions without degrading system responsiveness.

Solution

We combine message-bus back-pressure (prefetch limits), rate limiting at the API edge, and worker auto-scaling to smooth bursts while preserving SLAs for interactive users.

7 Testing Strategy

7.1 Frontend Testing

Frontend testing was performed manually as part of an end-to-end testing process, with a focus on the following aspects:

- Validating user interactions with the workflow canvas, including node creation and drag-and-drop functionality.
- Verifying correct handling of node connections and prevention of invalid workflows.
- Ensuring proper input validation and enforcement of required fields before workflow execution.
- Testing frontend interaction with backend services by triggering workflow execution requests.
- Confirming that the user interface accurately reflects workflow execution states and updates.

7.2 Unit Testing

Backend (pytest):

- API endpoints
- Service layer logic
- Authentication/authorization

Workers (Go testing):

- Node execution logic
- Context management
- Error handling and retry

7.3 Integration Testing

- End-to-end workflow execution
- Message queue integration
- WebSocket event delivery

7.4 AI Testing

Smith Evaluation:

- 86% valid workflow generation rate
- 79% successful execution without manual correction

Scryb Evaluation:

- 94% documentation accuracy

7.5 Performance Testing

- Simple workflows execute within seconds
- Linear scaling with node count
- Parallel execution improvements measured
- Low API latency maintained

8 Deployment & DevOps

8.1 Containerization

All runtime components are packaged as OCI/Docker images to standardize builds and deployments. The primary images used in this repository are:

- **rune-backend** — FastAPI backend (execution coordinator, secrets manager, API surface)
- **rune-worker** — Go-based worker engine (node execution runtime)
- **rune-websocket** (RTES) — Rust WebSocket server for real-time streaming
- **rune-frontend** — Next.js UI
- Supporting service images used in local and integration deployments: **postgres**, **redis**, **rabbitmq**, and **otel-collector**

Image build and tagging are performed as part of CI pipelines.

8.2 Docker Compose and Local Orchestration

The repository contains Docker Compose manifests for local development and simplified integration testing:

- **docker-compose.yml** — primary development compose file for bringing up API, worker, RTES, and supporting services
- **docker-compose.nginx.yml** — optional proxy/fronting configuration used for local HTTPS and reverse-proxy scenarios
- Service-specific compose snippets (for example under **services/***) used by component-level development and CI jobs

Compose usage supports network configuration, named volumes for stateful services, environment variable propagation (via **.env**), and dependency ordering for controlled start-up.

8.3 CI/CD Pipelines

Continuous integration and delivery are implemented via repository workflows that build, test, and publish artifacts and images. The repository includes CI workflows in the **.github/workflows** directory that cover the main components:

- Build and test jobs for **rune-backend**, **rune-worker**, **rune-websocket**, and **rune-frontend**
- Integration and e2e jobs that spin up compose stacks or ephemeral services to validate cross-component behavior

Notably, the repository contains a client-generation workflow (`generate-client.yml`) that runs the Hey API SDK generator: the generator configuration file `heyapi.conf.js` is used to produce client SDKs and API artifacts as part of the CI process. The CI job typically runs the generator, stores the generated SDK as a build artifact.

8.4 Building

Each component of RUNE is built as an independent Docker image.

8.4.1 Prerequisites

- Docker Desktop (or Docker Engine + Docker Compose v2)
- Git
- GNU Make (optional, for Makefile commands)

8.4.2 Environment Configuration

Before building or running containers, copy and configure environment variables:

```
cp .env.example .env
```

Then edit `.env` with your configuration settings.

8.4.3 Building with Makefile (Recommended)

The project includes a comprehensive Makefile for simplified build operations:

→ Start Rune

```
make up
```

→ Alternatively, start via nginx reverse proxy

```
make up-nginx
```

→ Rebuild and restart services

```
make restart
```

→ View running container status

```
make status
```

→ View logs from all services

```
make logs
```

8.4.4 Building with Docker Compose

Alternatively, Docker Compose commands can be used directly to build all services:

→ Build all images

```
docker compose build
```

→ Build and start in detached mode

```
docker compose up --build -d
```

→ Build with nginx proxy configuration

```
docker compose -f docker-compose.nginx.yml build
```

8.4.5 Cleanup

Remove containers, volumes, and images when needed:

→ Stop all services

```
make down
```

→ Full cleanup (removes volumes and images)

```
make docker-clean
```

9 Future Enhancements

9.1 Platform Features

- AI Agent Node (autonomous LLM-based execution)
- Collaborative editing
- Scheduled executions (cron-like)
- Webhook triggers
- Advanced analytics dashboard

9.2 Integrations

- OAuth2 integrations (Google, Slack, GitHub)
- CRM systems (Salesforce, HubSpot)
- Cloud storage (AWS S3, Google Drive)

9.3 Enterprise Features

- SAML SSO
- Advanced RBAC
- Audit logging
- Multi-tenancy

9.4 Performance

- Worker auto-scaling
- Database sharding
- CDN for frontend assets
- GraphQL API

10 Conclusion

RUNE demonstrates a pragmatic, production-oriented approach to AI-assisted workflow automation. The project delivers a clear separation between control (Backend Master), execution (Go Worker Engine), and real-time presentation (RTES in Rust), enabling low-latency streaming, horizontal scaling, and predictable throughput. Key outcomes include:

- A lightweight, auditable control plane that validates, persists, and dispatches executions.
- A fast, concurrent worker runtime for minimal execution delays and predictable performance.
- A real-time streaming layer that provides near-instant UI feedback with minimal jitter.
- Resilient messaging and storage patterns (RabbitMQ, Redis, PostgreSQL) that balance durability and responsiveness.

Looking forward, RUNE's modular architecture makes it straightforward to add new runtimes, improve AI-assisted generation, and integrate advanced observability or autoscaling strategies. The platform prioritizes reliability, low latency, and pragmatic operational simplicity to maximize real-world utility.

11 References

11.1 Project Resources

1. RUNE Project Repository

GitHub: <https://github.com/rune-org/rune>

Main source code repository containing all services and documentation.

11.2 Core Technologies

2. FastAPI Framework

<https://fastapi.tiangolo.com/>

Modern, high-performance web framework for building APIs with Python.

3. Go Programming Language

<https://go.dev/>

Official Go documentation and language specification.

4. Rust Programming Language

<https://www.rust-lang.org/>

Official Rust documentation and learning resources.

5. Next.js Framework

<https://nextjs.org/docs>

React framework for production-grade web applications.

6. React Flow Library

<https://reactflow.dev/>

React-based library for building interactive, node-based editors and workflow visualization interfaces.

7. PostgreSQL Database

<https://www.postgresql.org/docs/>

Official PostgreSQL documentation covering SQL, administration, and internals.

8. Redis Documentation

<https://redis.io/docs/>

In-memory data structure store documentation including pub/sub and caching patterns.

9. RabbitMQ Documentation

<https://www.rabbitmq.com/documentation.html>

Message broker documentation covering AMQP protocol and queue management.

11.3 Development Tools

10. Docker Documentation

<https://docs.docker.com/>

Container platform documentation for building and deploying applications.

11. Docker Compose

<https://docs.docker.com/compose/>

Tool for defining and running multi-container applications.

12. Pydantic

<https://docs.pydantic.dev/>

Data validation library using Python type annotations.

13. Hey API OpenAPI Generator

<https://heyapi.dev/>

TypeScript client SDK generator from OpenAPI specifications.

14. Alembic

<https://alembic.sqlalchemy.org/>

Database migration tool for SQLAlchemy.

11.4 Observability and Monitoring

15. OpenTelemetry

<https://opentelemetry.io/docs/>

Observability framework for cloud-native software.

Acknowledgments

We would like to express our sincere gratitude to our supervisors and mentors, **Dr. Yomna Safaaeldin** and **Eng. Amr Ibrahim**, for their continuous support, technical guidance, and mentorship. Additional thanks to:

- Project supervisors and examiners for their feedback and review.
- The development team: Shehab Mahmoud, Abdelrahman Hany, Youssef Shahean, Seif Tamer, Seif Elwarwary, Omar Mamon, Habiba El Sayed, Zeynat Haytham, and Shahd Ashraf for implementation and documentation.
- Open-source tool maintainers and communities (FastAPI, Go, Rust, RabbitMQ, PostgreSQL, Redis) whose libraries and tooling made this project possible.

Their combined support and contributions were essential to deliver the RUNE platform.

Appendices

Appendix A: API Reference

This appendix points to the authoritative API specification and highlights the primary endpoints used by clients and CI jobs.

Key artifacts:

- Full OpenAPI specification: `services/api/openapi.yaml`
- Generated frontend client: `apps/web/src/hey-api.ts` (produced by the Hey API generator)
- Common endpoints: authentication (`/auth`), workflows (`/workflows`), executions (`/executions`), status (`/status`), and WebSocket token issuance (`/ws/token`).

To regenerate the client locally (from repo root):

```
pnpm dlx @hey-api/openapi-ts -f heyapi.conf.js
```

Appendix B: Message Schemas

This appendix documents the primary message types used across RabbitMQ, Redis pub/sub, and WebSocket channels.

Primary schemas (summary):

- **NodeExecutionMessage**: `execution_id`, `node_graph`, `node_id`, `inputs`, `credential_ref`, `policy`
- **NodeStatusMessage**: `execution_id`, `node_id`, `status`, `timestamp`, `short_log`
- **CompletionMessage**: `execution_id`, `status`, `result`, `artifacts_refs`, `duration`
- **TokenMessage**: ephemeral credential or lease deliveries for workers (encrypted payload, short TTL)

The canonical schemas live alongside the service code; see `services/api/openapi.yaml` and the messaging package in the worker and backend code for full definitions. Messages intended for UI streaming are intentionally compact to minimize bandwidth and serialization overhead.

Appendix C: Database Schema

This appendix summarizes the primary database tables and where to find the migration history.

Primary tables:

- **workflows** — stored workflow definitions (JSONB), metadata, owner, and schema version
- **executions** — execution records, status, start/end timestamps, and result references
- **users** — user accounts, roles, and authentication metadata
- **credentials** — encrypted credential blobs, metadata, and rotation info
- **artifacts** — references to stored artifacts and their storage locations

Migration files and schema history are in `services/api/migrations`. Use standard migration tooling (Alembic) from the backend service to inspect or apply schema changes.

Appendix D: Configuration Reference

Common environment variables and their roles:

- `DATABASE_URL` — PostgreSQL connection string
- `RABBITMQ_URL` — AMQP connection (`user:pass@host:port/`)
- `REDIS_URL` — Redis connection URL
- `SECRET_KEY` — Application-level encryption and signing key
- `OTEL_COLLECTOR_URL` — Optional OpenTelemetry collector endpoint
- `WEB_BASE_URL` — Frontend base URL used for generated links and callbacks
- `LOG_LEVEL` — Runtime logging verbosity (`info`, `debug`, `warn`, `error`)

Keep secrets and credentials in CI secrets or a vault; do not commit them to source control.

Appendix E: Deployment Guide

Minimal local deployment steps (development):

1. Ensure Docker and Docker Compose are installed.
2. From the repo root, bring up the development stack:

```
docker-compose up --build -d
```

3. Run backend tests (example):

```
cd services/api
pytest -q
```

4. Generate client SDK after API changes:

```
pnpm dlx @hey-api/openapi-ts -f heyapi.conf.js
```

Troubleshooting tips:

- Check service logs via `docker-compose logs -f <service>` for runtime errors.
- Verify connectivity strings (`DATABASE_URL`, `RABBITMQ_URL`, `REDIS_URL`) if services fail to connect.
- Use the included CI workflows as a reference for production image build and tag steps.